

18TH AFRICAN CONFERENCE ON RESEARCH IN COMPUTER SCIENCE AND APPLIED MATHEMATICS

CARI 2026 · Cotonou, Benin · 22 October 2026

VulHGT

Leveraging Code Property Graphs and Heterogeneous Graph Transformers for PHP Vulnerability Detection

Sèdofi Hope Lanmayikpohoue[1], Emery Kouassi Assogba[2], Mahugnon Géraud Azehoun Pazou[1], John Oscar Raoul Aoga[3], Miton Abel Konnon[1]

[1]National University of Sciences, Technologies, Engineering and Mathematics (UNSTIM), Abomey, Benin

[2]Institute of Mathematics and Physical Sciences (IMSP), Dangbo, Benin

[3]UCLouvain, Belgium

OUTLINE

What this talk covers

01 **Motivation**
Why PHP, why XSS and SQLi, why now

02 **Related work**
Static analysers, sequential models, homogeneous GNNs

03 **The VulHGT framework**
Slicing, node embeddings, heterogeneous attention

04 **Results**
Detection, ablations, SAST baselines, real code

05 **Limitations**
What the study does not yet establish

06 **Conclusion and future work**
Line-level detection, recall-focused training

MOTIVATION

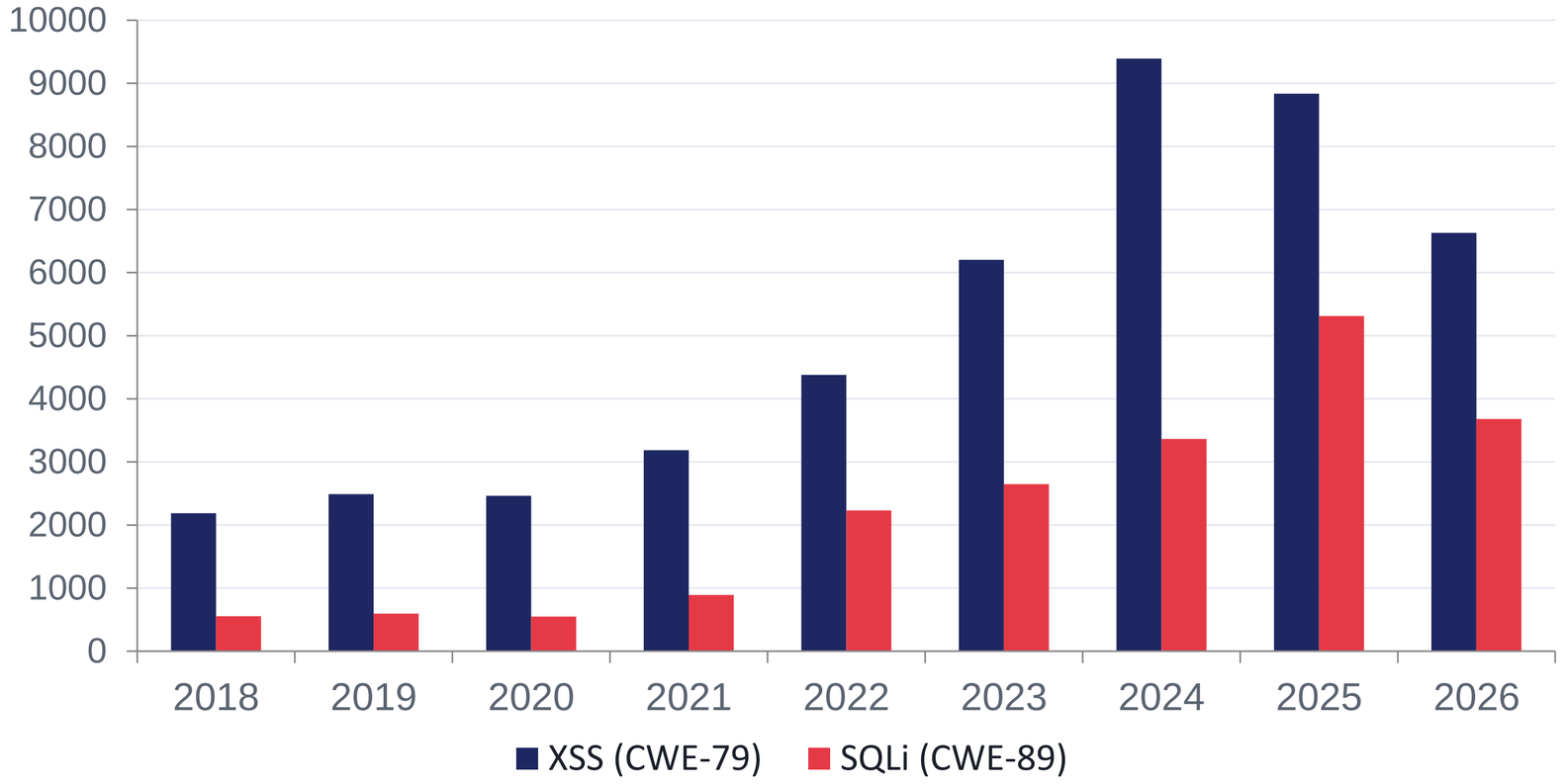
Frequent, severe and concentrated in PHP

74%

of all websites run on PHP

Both classes have been documented for more than twenty years, yet the yearly counts show no decline. These are not legacy bugs on their way out: fresh code reintroduces them at every release cycle. PHP still serves most of the web, so that steady stream meets the widest attack surface of any server-side language, and detection tooling has not kept up.

Reported CWE-79 (XSS) and CWE-89 (SQLi) per year, NIST NVD (20 September 2026)



RELATED WORK

Where existing detection approaches stop

01

Rule based SAST

RIPS · Pixy · WAP · Progpilot

Fixed patterns cannot cover PHP dynamic typing and the combinatorial diversity of real data-flows.

Best F1 below 0.60

02

Sequential deep learning

LSTM and Bi-LSTM over tokens

Code is read as linear text, so control-flow and data-dependency structure are discarded.

Fragile to renaming

03

Homogeneous GNNs

DeepTective · VulEye · Dtector

Every node and edge type is aggregated identically, erasing the semantics a CPG encodes.

No type-awareness

A heterogeneous GNN over the full code property graph is the natural next step, and it remains barely explored for PHP.

OUR PROPOSAL

VulHGT: Code property graphs plus heterogeneous graph transformers

01 **Adaptive source-to-sink CPG slicing**

Focuses each function graph on the relevant taint path, cutting noise and extraction cost.

02 **GraphCodeBERT node initialisation**

Every node carries a 768-dimensional embedding of what the code does, not only how it is written.

03 **Type-aware heterogeneous graph transformer**

Three layers, eight heads, separate attention per meta-relation over AST, CFG, DDG and CDG.

04 **Jumping Knowledge aggregation**

Concatenates all layer outputs, so local motifs survive depth alongside long range dependencies.

05 **Multi-perspective global pooling**

Mean, sum and attention pooling concatenated into a single graph-level representation.

METHOD

What a code property graph holds

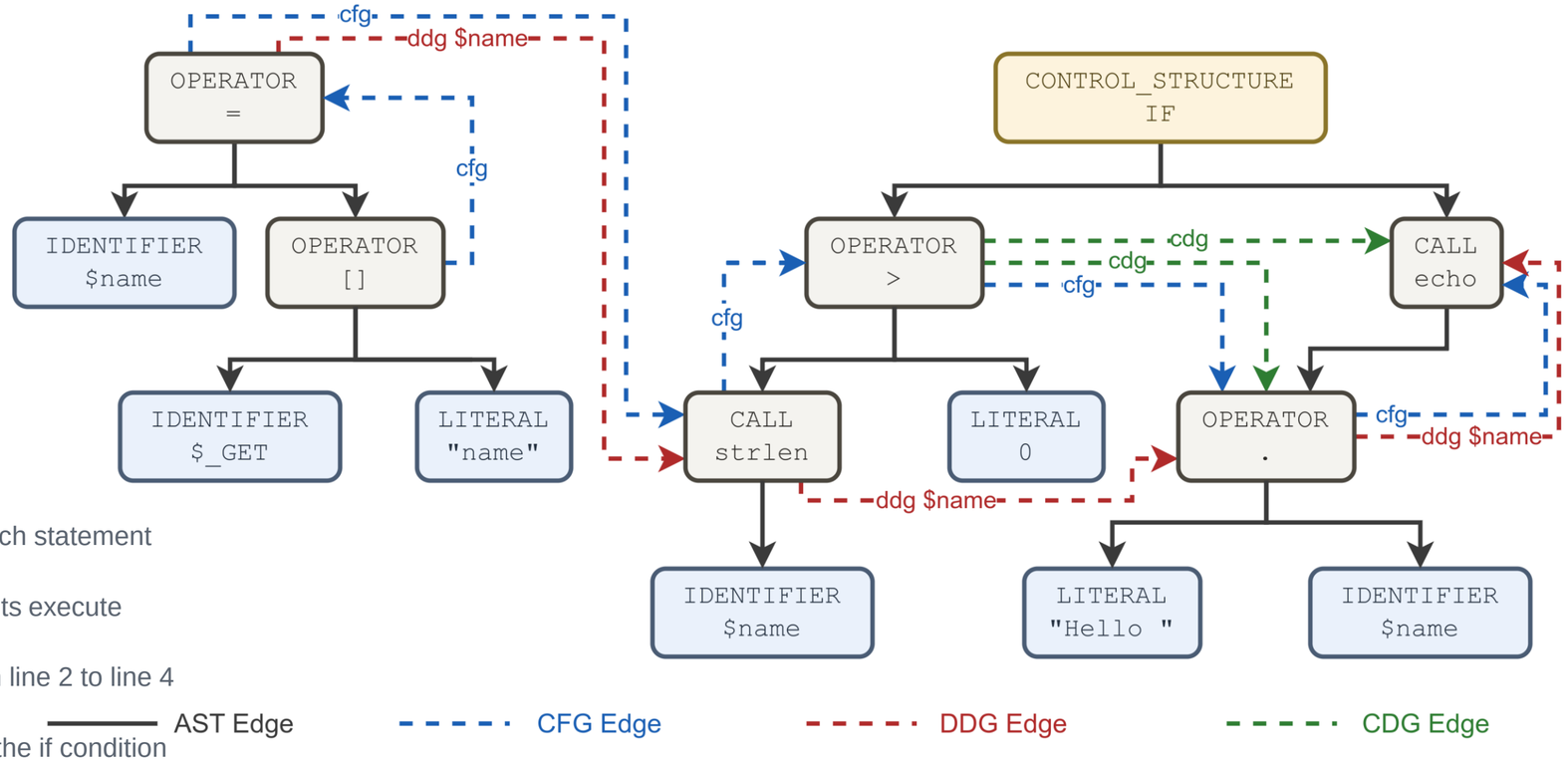
```
1  <?php
2  $name = $_GET['name'];
3  if (strlen($name) > 0) {
4      echo "Hello " . $name;
5  }
6  ?>
```

Line 2 untrusted input enters, the source.

Line 4 it reaches output unsanitised, the sink.

THE FOUR RELATIONS

- **Abstract Syntax Tree** — syntactic structure of each statement
- **Control Flow Graph** — order in which statements execute
- **Data Dependency Graph** — how $name travels from line 2 to line 4
- **Control Dependency Graph** — the echo is guarded by the if condition

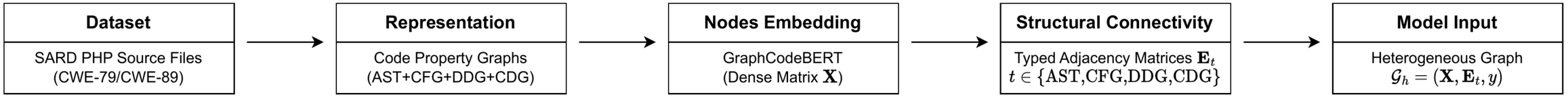


Simplified code property graph of the snippet, as extracted by Joern.

The AST defines the statement nodes themselves, and a CPG merges all four relations into one graph. VulHGT attends over that fusion type by type.

METHOD

From PHP source to heterogeneous graph



Adaptive slicing, inside the representation step

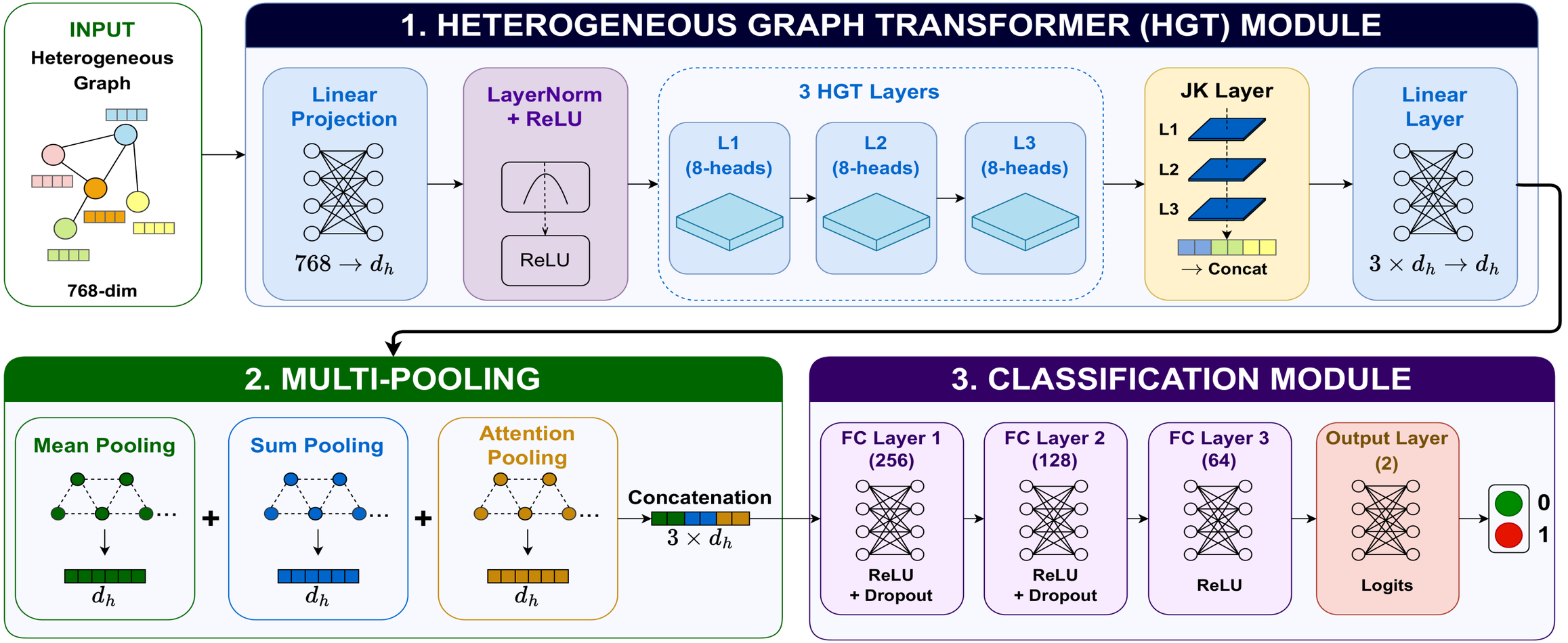
A 100-line PHP file can yield more than a thousand raw nodes and edges, most of them unrelated to any vulnerability. We keep the subgraph bounded between the first taint source and the last sink, and fall back to the full function graph when no clear path is found.

What reaches the model

Smaller, focused subgraphs that still carry all four edge types. Sources are inputs such as $_GET and $_POST, sinks are sensitive calls such as echo, mysqli_query and PDO exec. The classification unit is the file.

METHOD

Model architecture



d_h: Hidden dimension FC: Fully Connected JK: Jumping Knowledge

EXPERIMENTAL SETUP

Dataset, splits and metrics

SARD Dataset (CWE-79 and CWE-89)

Vulnerability	Safe	Vulnerable	Total
CWE-79 (XSS)	130,809 (82.1%)	28,555 (17.9%)	159,364
CWE-89 (SQLi)	74,377 (78.7%)	20,150 (21.3%)	94,527

Stratified 80 / 10 / 10 train, validation and test split. Comments stripped before extraction. The test set stays isolated until the final evaluation.

Two compute environments

Training on a local Intel Xeon Silver 4114 server with 128 GB of RAM. Inference timings on a NVIDIA P100.

Evaluation metrics

- Accuracy, precision, recall, F1 on every task.
- AUC-PR (Precision-Recall) rather than AUC-ROC, since it does not reward performance on the majority (safe) class in imbalanced data.
- F1 is the primary metric: it balances the cost of missed vulnerabilities (false negatives) against alert fatigue from false positives.

DETECTION PERFORMANCE

State-of-the-art results on SARD

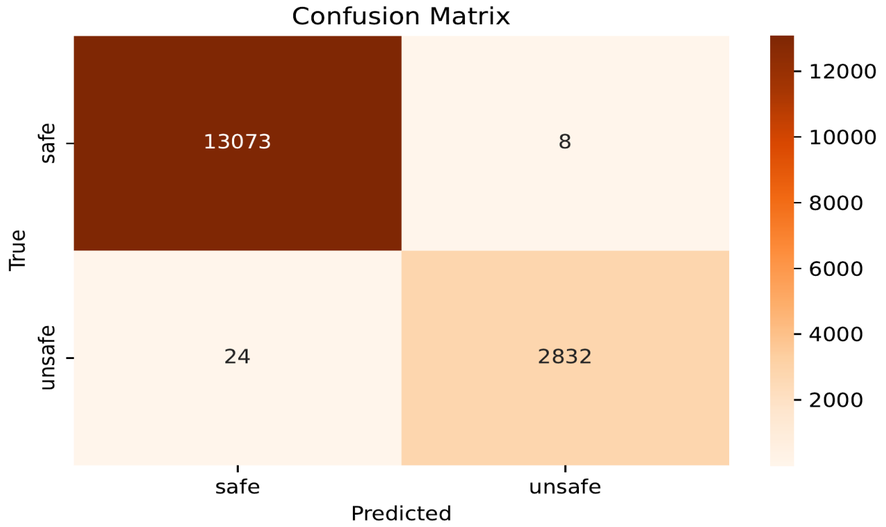
99.44%

XSS F1-score

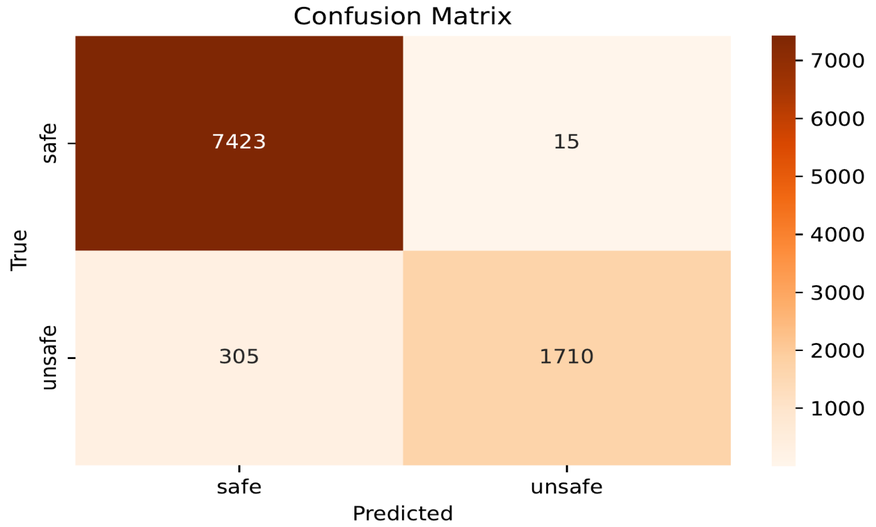
91.44%

SQLi F1-score

Task	Precision	Recall	AUC-PR
XSS	99.72%	99.16%	0.99986
SQLi	99.13%	84.86%	0.97831



XSS Confusion Matrix

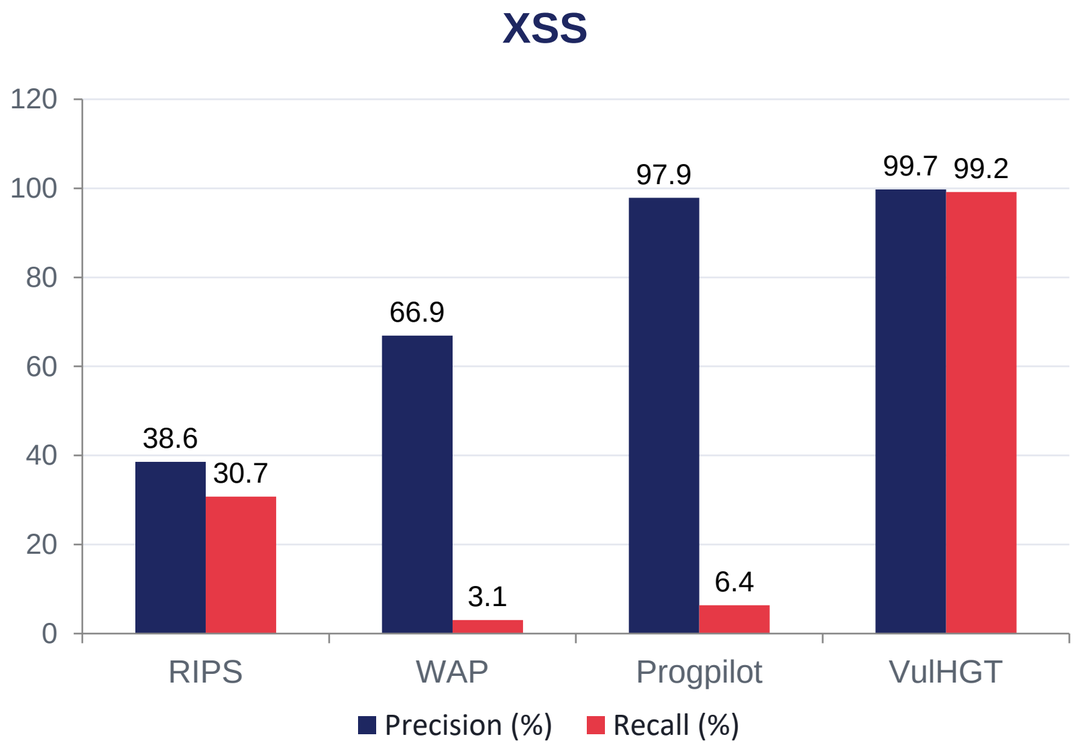


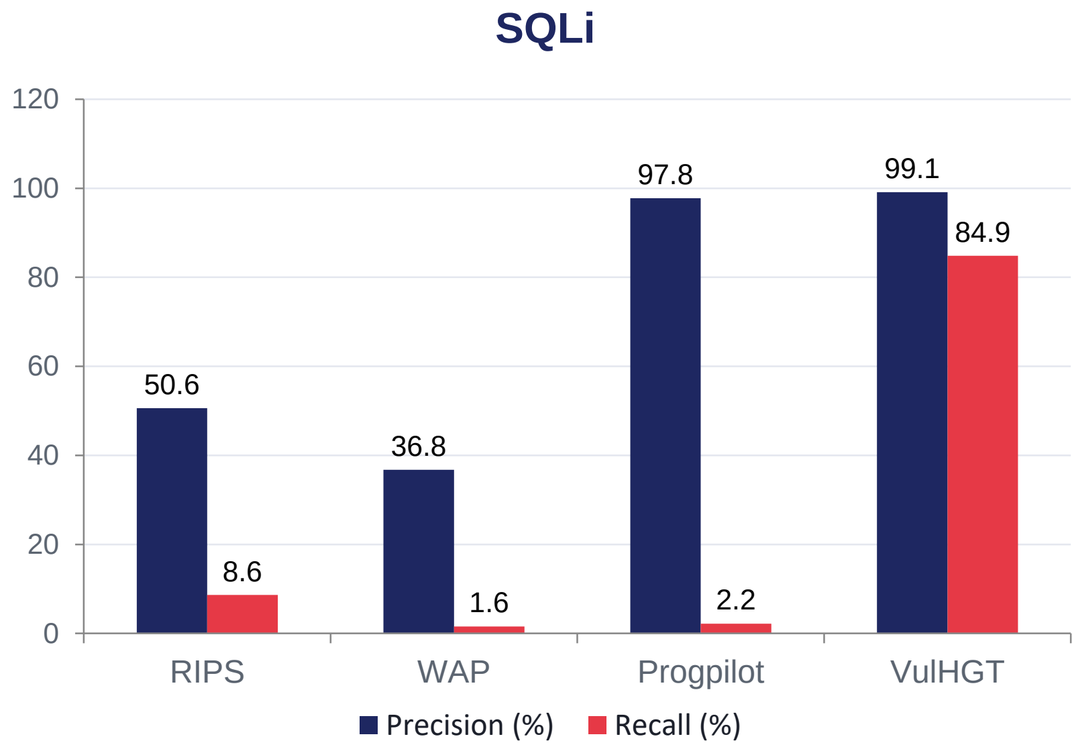
SQLi Confusion Matrix

The SQLi main gap is lower recall, not precision: 305 false negatives against 15 false positives. The model is conservative rather than noisy, rarely wrong when it flags a file, but misses more subtle injection patterns.

COMPARISON WITH SAST TOOLS

Rule-based tools collapse on recall



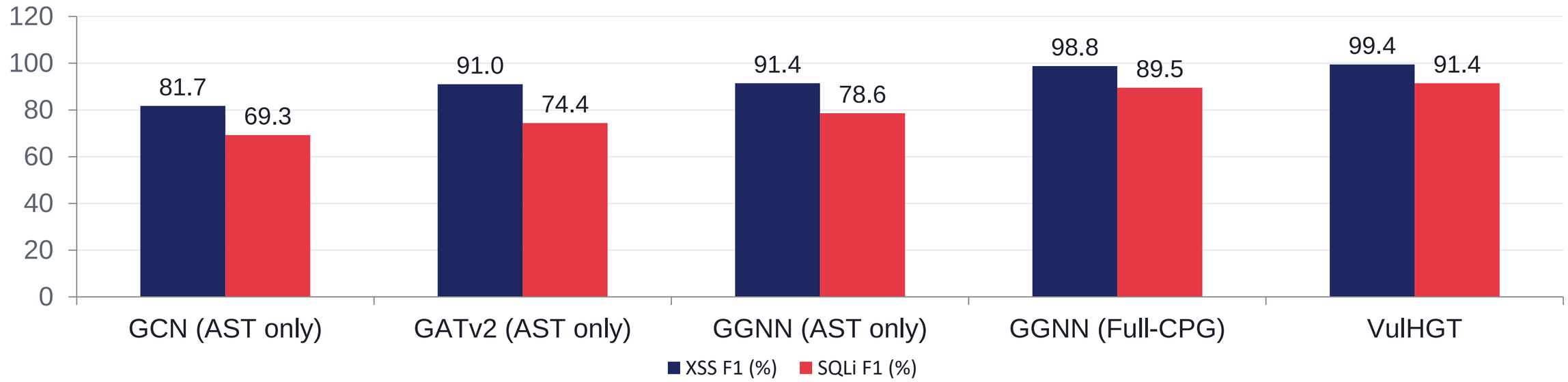


Progpilot reaches 97-98% precision on both tasks but recovers only 6.37% (XSS) and 2.18% (SQLi) of the vulnerabilities. Predefined patterns cannot cover PHP data-flow diversity.

ABLATION STUDY

Isolating heterogeneous attention from graph coverage

Does the gain come from seeing more of the graph, or from type-aware attention itself? GGNN (Full-CPG) receives the same four edge types as VulHGT and differs only in how it aggregates them.



Most of the gain is graph coverage

DDG and CDG edges add 7.36 F1 points on XSS and 10.91 on SQLi to the same GGNN homogeneous model.

Heterogeneity buys precision

On identical graphs, type-aware attention adds 0.66 and 1.95 F1 points, mostly through SQLi precision.

ABLATION STUDY

GraphCodeBERT embeddings are necessary, not incidental

XSS F1 score

99.44% → **26.83%**

with GraphCodeBERT → random features

72.6 points lost

SQLi F1 score

91.44% → **28.67%**

with GraphCodeBERT → random features

62.8 points lost

Same topology, same HGT, Jumping Knowledge and pooling. Only the node embedding source changes. This isolates GraphCodeBERT as the primary driver of VulHGT's discriminative power: the model has to know what a node represents, a tainted variable or a sanitisation call, not merely how nodes are connected.

REAL-WORLD GENERALISATION

Beyond SARD: lab benchmarks and a real application

Dataset	Task	Files	VulHGT recall	VulHGT precision	VulEye recall	Dtector recall
XSS-Labs	XSS	18	**100.00%**	n/a	72.22%	88.88%
SQLi-Labs	SQLi	69	**91.30%**	n/a	88.41%	89.85%
CloudClassroom	XSS	57	**100.00%**	64.90%	n/a	n/a
CloudClassroom	SQLi	57	**94.10%**	58.20%	n/a	n/a

Why precision drops on real code

Distribution mismatch: SARD safe files are patched variants of vulnerable ones. Real applications contain structurally different safe files, templates, logout scripts, configuration headers, which lack the data-flow signatures the model learned to read as safe.

The operational trade-off

For a security tool, high recall at the cost of some false positives is preferable to missing a real vulnerability. Ground truth on CloudClassroom comes from manual review of every data-flow.

DISCUSSION

Limitations, stated openly

File-level, not line-level

Binary classification per file. It does not yet pinpoint the vulnerable line for rapid remediation.

Two components not ablated

Jumping Knowledge and multi-view pooling follow established designs and stay fixed across variants, so their individual share is unknown.

One real application benchmark

CloudClassroom, 57 files. Broader ecosystems such as WordPress and Laravel are desirable, but manual ground truth does not scale.

Unified hyperparameters

The same configuration serves both tasks by design, which likely leaves headroom on SQLi at F1 91.44%.

CONCLUSION

Contributions and what comes next

WHAT VULHGT DELIVERS

01 State-of-the-art F1 on SARD: 99.44% on XSS, 91.44% on SQLi

02 Outperforms rule-based SAST tools and homogeneous GNN baselines

03 Strong recall on genuine, real-world PHP applications

04 CI/CD viable: a 57-file project is scanned end-to-end in under 45 seconds

FUTURE WORK

Line-level detection derived from HGT attention weights

Task-specific tuning and recall-focused loss weighting for SQLi

Component-wise ablation of Jumping Knowledge and multi-view pooling

Thank you

Questions and discussion

Sèdofi Hope Lanmayikpohoue

UNSTIM, Abomey, Benin

slanmayikpohoue@unstim.bj